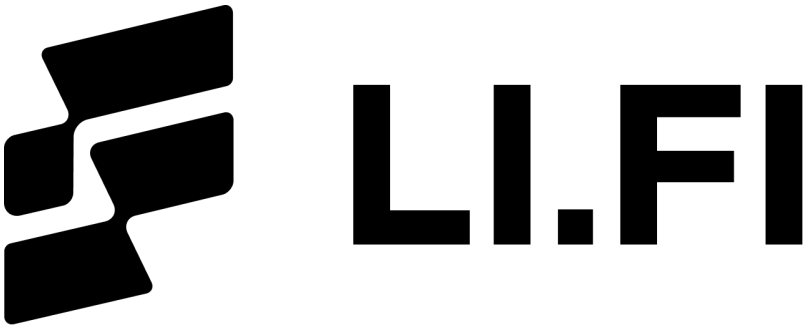

LI.FI Security Review

PolymerCCTPFacet.sol(v3.0.0)

Security Researcher

Sujith Somraaj (somraajsujith@gmail.com)

Report prepared by: Sujith Somraaj

July 16, 2026

Contents

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the Web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

- src/Facets/PolymerCCTPFacet.sol(v3.0.0)
- src/Helpers/LiFiData.sol(v1.0.1)
- src/Interfaces/ITokenMessenger.sol(v1.1.0)

4 Risk classification

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

High leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.

Medium global losses <10% or losses to only a subset of users, but still unacceptable.

Low losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

High almost certain to happen, easy to perform, or not easy but highly incentivized

Medium only conditionally possible or incentivized, but still relatively likely

Low requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Critical Must fix as soon as possible (if already deployed)

High Must fix (before deployment if not already deployed)

Medium Should fix

Low Could fix

5 Executive Summary

Over the course of 6 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

In this period of time a total of 5 issues were found.

Project Summary	
Project Name	LI.FI
Repository	lifinance/contracts
Commit	45e319a2
Audit Timeline	July 15, 2026 - July 16, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found	
Critical Risk	0
High Risk	0
Medium Risk	3
Low Risk	1
Gas Optimizations	0
Informational	1
Total Issues	**5**

6 Findings

6.1 Medium Risk

6.1.1 Solana destinations accept EVM-style receivers, producing normally non-executable CCTP burns

Context: PolymerCCTPFacet.sol#L313-L359, PolymerCCTPFacet.sol#L379-L409

Description: `_startBridge()` never rejects a normal, non-sentinel `BridgeData.receiver` when `destinationChainId` is Solana. With `destinationChainId = LIFI_CHAIN_ID_SOLANA`, a normal EVM `receiver`, and empty `hookData`, the call skips the HyperCore and Stellar corridor branches, passes the generic hook-data check, and enters the normal-receiver mint branch:

```
TOKEN_MESSENGER.depositForBurn(
    bridgeAmount,
    domainId,                                          // Solana CCTP domain 5
    bytes32(uint256(uint160(_bridgeData.receiver))),   // zero-padded EVM address
    USDC,
    UNRESTRICTED_DESTINATION_CALLER,
    _polymerData.maxCCTPFee,
    _polymerData.minFinalityThreshold
);
```

Solana's CCTP domain `5` is configured, so `_chainIdToDomainId()` does not revert and the source burn succeeds. However, Circle requires the Solana `mintRecipient` to be the recipient's USDC Associated Token Account. A zero-padded ordinary EVM address is neither derived nor validated as such an account. The resulting CCTP message therefore normally cannot execute on Solana, leaving the source USDC burned and the message unconsumed.

The existing zero-address check does not prevent this outcome because it only rejects `address(0)`. The facet can determine that Solana requires the sentinel solely from `destinationChainId`; no EVM-side proof of Solana account ownership is required.

Recommendation: Before any swap or token transfer, reject a non-sentinel receiver for Solana:

```
if (
    _bridgeData.destinationChainId == LIFI_CHAIN_ID_SOLANA &&
    _bridgeData.receiver != NON_EVM_ADDRESS
) {
    revert InvalidReceiver();
}
```

Apply the same invariant to any future non-EVM destination whose recipient cannot be represented by a normal EVM address. Stellar already enforces this direction in its corridor-specific validation.

LI.FI: Fixed in c7ca594

Researcher: Verified fix.

6.1.2 Swap refunds routed through Permit2Proxy are sent to the shared proxy instead of the user

Context: PolymerCCTPFacet.sol#L287, PolymerCCTPFacet.sol#L293

Description: `swapAndStartBridgeTokensViaPolymerCCTP()` sends both excess native currency and source-swap leftovers to `msg.sender`:

```
refundExcessNative(payable(msg.sender))

_bridgeData.minAmount = _depositAndSwap(
    _bridgeData.transactionId,
    _bridgeData.minAmount,
    _swapData,
    payable(msg.sender)
);
```

When the function is invoked through `Permit2Proxy`, the proxy calls the Diamond with a normal external call. Consequently, `msg.sender` inside the facet is the shared proxy contract rather than the permit signer or intended refund recipient. `Permit2Proxy` returns only the Diamond's return data after the call and does not forward native currency or ERC-20 balances received during execution to the signer.

Excess native currency therefore remains in `Permit2Proxy` and can only be recovered through its owner-only withdrawal function. ERC-20 leftovers have a more severe outcome: they may be claimed by a later proxy user rather than merely remaining administratively recoverable.

Before calling the Diamond, `Permit2Proxy` grants it a maximum allowance for the permitted token. The amount that the Diamond subsequently pulls from the proxy is determined by the user-supplied Diamond calldata and is not bounded to the amount newly transferred into the proxy by that invocation's permit. Therefore, after a victim's leftover token is refunded to the shared proxy, another user can:

1. authorize a small amount of the same token through Permit2;
2. submit Diamond calldata whose deposit amount includes the proxy's pre-existing balance; and
3. route the resulting swap or bridge output to themselves.

The later call is authorized for the attacker's own permit and calldata, but it spends a balance that belongs to the earlier user because the proxy does not maintain per-user accounting. Depending on the asset and available Diamond route, the victim's ERC-20 leftovers can therefore be permissionlessly extracted.

Recommendation: Add an explicit `address refundRecipient` to `PolymerCCTPData`. In the swap entrypoint, reject `address(0)` and use the field for both refund paths:

```
if (_polymerData.refundRecipient == address(0)) {
    revert InvalidReceiver();
}
```

```
refundExcessNative(payable(_polymerData.refundRecipient))
```

```
_bridgeData.minAmount = _depositAndSwap(
    _bridgeData.transactionId,
    _bridgeData.minAmount,
    _swapData,
    payable(_polymerData.refundRecipient)
);
```

The API should set `refundRecipient` to the end user, not to `msg.sender`, because `msg.sender` may be `Permit2Proxy` or another intermediary. As defense in depth, `Permit2Proxy` should also avoid retaining user balances and should ensure that a Diamond invocation cannot spend more of a token than was supplied for that invocation.

LI.FI: Fixed in 92b2a7e

Researcher: Verified fix.

6.1.3 Non-EVM sentinel can be used for EVM destinations, decoupling the declared receiver from the CCTP mint recipient

Context: PolymerCCTPFacet.sol#L430

Description: `_startBridge()` validates the `NON_EVM_ADDRESS` sentinel within the HyperCore and Stellar corridor branches, but the generic sentinel mint branch does not verify that the selected destination actually uses a non-EVM receiver representation. Consequently, the sentinel is accepted for any configured EVM destination.

For example, consider the following parameters:

```
destinationChainId = 8453; // Base
receiver = NON_EVM_ADDRESS;
nonEVMReceiver = bytes32(uint256(uint160(attacker)));
hookData = "";
```

The call skips the HyperCore and Stellar branches, passes the empty-hook check, and reaches the generic sentinel branch. Since Base is not Solana, the facet selects `nonEVMReceiver` as the CCTP `mintRecipient`:

```
bool isSolanaDestination = destinationChainId == LIFI_CHAIN_ID_SOLANA;

bytes32 mintRecipient = isSolanaDestination
    ? _polymerData.solanaReceiverATA
    : _polymerData.nonEVMReceiver;

TOKEN_MESSENGER.depositForBurn(
    bridgeAmount,
    domainId,
    mintRecipient,
    USDC,
    UNRESTRICTED_DESTINATION_CALLER,
    _polymerData.maxCCTPFee,
    _polymerData.minFinalityThreshold
);
```

Base has a configured CCTP domain, so the source burn succeeds. Circle's EVM `AddressUtils` converts the `bytes32` mint recipient to an EVM address using its low 20 bytes, causing the destination USDC to be minted to `attacker`.

At the same time, the canonical recipient recorded in `BridgeData.receiver` and emitted through `LiFiTransferStarted` remains the fixed sentinel. Tooling that treats `BridgeData.receiver` as the destination beneficiary can therefore display or index a different recipient from the address that receives the CCTP mint. Although `nonEVMReceiver` is part of the authorized calldata, the destination/receiver combination is deterministically invalid and can be rejected on-chain without attempting to prove ownership of any destination account.

Recommendation: When `BridgeData.receiver` is the sentinel, allow only destinations whose routing explicitly requires that representation. For the destinations currently supported by this facet:

```
if (_bridgeData.receiver == NON_EVM_ADDRESS) {
    bool supportsSentinel =
        _bridgeData.destinationChainId == LIFI_CHAIN_ID_SOLANA ||
        _bridgeData.destinationChainId == LIFI_CHAIN_ID_STELLAR;

    if (!supportsSentinel) {
        revert InvalidReceiver();
    }
}
```

Perform this validation before swaps or token transfers. HyperCore must not be included in `supportsSentinel`, because its forwarding corridor intentionally requires a normal EVM-shaped `BridgeData.receiver` that is bound to the receiver encoded in `hookData`.

LI.FI: Fixed in c7ca594

Researcher: Verified fix.

6.2 Low Risk

6.2.1 Unsupported Stellar hook versions and empty recipients can produce burns that the destination forwarder cannot execute

Context: PolymerCCTPFacet.sol#L341-L346

Description: Stellar transfers mint to Circle's pinned `CctpForwarder`, which extracts the final Stellar `strkey` recipient from `hookData`. Circle's documented Stellar hook layout is:

```
bytes  0-23: reserved bytes
bytes 24-27: uint32 version, currently 0
bytes 28-31: uint32 recipient length L
bytes 32... : L bytes containing the G-, C-, or M-prefixed Stellar strkey
```

The facet verifies that `hookData.length >= 32` and that the declared recipient length equals the bytes remaining after the 32-byte header. It does not validate the 32-bit version field, and it accepts `L == 0` when the total hook length is exactly 32 bytes.

Circle's source-side `TokenMessengerV2` treats hook data as opaque bytes, so neither condition prevents `depositForBurnWithHook()` from burning the source USDC and emitting an attested CCTP message. On Stellar, however, the current forwarder supports version `0` and requires a valid non-empty recipient `strkey`. An unsupported version or empty recipient causes `mint_and_forward` to revert.

The Stellar mint and forward occur atomically, so a failed destination invocation does not leave USDC minted in the forwarder. Instead, the source USDC remains burned and the CCTP message remains unconsumed. Because the malformed hook is part of the attested message, the transfer cannot be retried with corrected hook bytes and remains non-executable under the supported forwarder behavior.

HyperCore has an analogous version field, but its omission is already documented as a deliberate reliance on trusted LI.FI API calldata. That acknowledged HyperCore trust decision is not part of this finding.

Recommendation: Validate the Stellar version and require a non-empty declared recipient before calling `depositForBurnWithHook()`:

```
uint32 version = uint32(bytes4(_polymerData.hookData[24:28]));
uint256 recipientLength = uint32(
    bytes4(_polymerData.hookData[28:32])
);

if (version != 0 || recipientLength == 0) {
    revert InvalidCallData();
}
```

Also align the first 24 reserved bytes used by the API and test fixtures with Circle's current Stellar specification. If the destination forwarder requires a specific value, enforce that value before burning as well.

LI.FI: The `strKey` length validation is updated to not allow empty recipients in b81c039. Version check is not done intentionally and is documented in the same commit.

Researcher: Verified fix and acknowledged the documentational update.

6.3 Informational

6.3.1 Swap output asset is not constrained to USDC, allowing pre-existing Diamond USDC to be bridged

Context: PolymerCCTPFacet.sol#L293

Description: `swapAndStartBridgeTokensViaPolymerCCTP()` verifies that `BridgeData.sendingAssetId` is USDC through `onlyAllowSourceToken`, but it does not verify that the final swap's `receivingAssetId` is USDC.

Consequently, a caller can construct an allowed swap whose final output is another token. If the Diamond has a pre-existing USDC balance, the swap output amount can be treated as a USDC amount and that existing USDC

can be burned and minted to the caller's destination receiver. The unrelated final swap output remains in the Diamond.

Without a pre-existing USDC balance, the TokenMessenger transfer normally reverts, but the Diamond may hold USDC from accidental transfers, rounding, refunds, or other integrations. Those shared balances should not be spendable through user-controlled swap calldata.

Recommendation: Before depositing tokens or executing swaps, require the final swap output asset to equal USDC:

```
if (
    _swapData.length > 0 &&
    _swapData[_swapData.length - 1].receivingAssetId != USDC
) {
    revert InvalidSendingToken();
}
```

LI.FI: Fixed in f2b70b3

Researcher: Verified fix.